

Components for Sorting Network Proofs

A catalogue of the constructions used in the verification of djbsort

Laurent Théry

Abstract. This note catalogues the constructions used in the two verifications of djbsort — the portable `int32` sort and the AVX2 sort — in the style of *A Formalisation of Algorithms for Sorting Network*. It records what each component is, why it exists, and which of them are shared. Components introduced since that note are marked **NEW**.

1 Introduction

A network is a sequence of connectors acting on m lines. The original note develops that model and three algorithms that build sorting networks: the bitonic sorter, Batcher’s odd-even merge sorter, and Knuth’s odd-even exchange sorter. This note starts where that one stops.

Verifying real code needed more than the model. Two programs were verified:

- `int32` — djbsort’s portable `sort.c`, which implements Knuth’s merge exchange (TAOCP 5.2.2M) as a flat loop nest;
- `avx2` — djbsort’s vectorised sort, whose 8×8 transpose and sign-flip masks realise a periodic bitonic network.

Each raised a different difficulty, and each grew its own machinery. Section 2 recalls the foundation. Section 3 describes the components that are generic and now shared. Sections 4 and 5 describe what remains specific to each track. Section 6 summarises status and file layout.

The organising observation is that the foundation offers only **binary** combinators — everything is built by splitting a network in halves. Both verifications needed to say instead “run this small network **everywhere**”, on blocks or on residue classes, and both had to build that themselves.

2 The foundation

2.1 Connectors and networks

A connector links each line to a partner, involutively, and records for each line whether the comparison is flipped.

```
Record connector (m : nat) := connector_of {
  clink : {ffun 'I_m -> 'I_m};
  cflip : {ffun 'I_m -> bool};
  cfinv : [forall i, clink (clink i) == i];
  cflipinv : [forall i, cflip (clink i) == cflip i] }.

Definition cfun c t :=
  [tuple let min := min (tnth t i) (tnth t (clink c i)) in
    let max := max (tnth t i) (tnth t (clink c i)) in
    if i <= clink c i then if cflip c i then max else min
    else if cflip c i then min else max | i < m].

Definition network := seq (connector m).
Definition nfun n t := foldl (fun t c => cfun c t) t n.
Definition sorting := [qualify n | [forall r : m.-tuple bool, sorted <=%0 (nfun n r)]].
```

A connector is one **parallel stage**: it performs several disjoint comparators at once. This is worth stressing, because it is exactly where the `int32` track ran into trouble (§4.2).

2.2 Building networks

The combinators available are all binary — they split $m + m$ lines into two halves, either contiguous or by parity.

<code>cmerge, cdup</code>	contiguous halves, at connector level
<code>ceomerge, ceodup</code>	even/odd halves, at connector level
<code>nmerge, ndup</code>	contiguous halves, at network level
<code>neomerge, neodup</code>	even/odd halves, at network level
<code>cswap i j</code>	a single comparator between lines i and j
<code>codd_jump k</code>	odd lines linked to their k -th successor (k odd)
<code>ceswap</code>	even lines linked to their odd neighbour

The semantics of the parity split is the one identity everything else rests on:

```
Lemma nfun_eodup (n : network m) (t : (m + m).-tuple A) :
  nfun (neodup n) t = teocat (nfun n (tetake t)) (nfun n (totake t)).
```

2.3 The three sorters

```
Fixpoint half_cleaner_rec b m : network (`2^ m) := ... (* bitonic *)
Fixpoint bsort m : network (`2^ m) := ...
Fixpoint bfsort (b : bool) m : network (`2^ m) := ...

Definition batcher_merge {m} : connector m := codd_jump 1. (* odd-even merge *)
Fixpoint batcher m : network (`2^ m) :=
  if m is m1.+1 then ndup (batcher m1) ++ batcher_merge_rec m1.+1 else [::].

Fixpoint knuth_jump_rec m k r : network m :=
  (* odd-even exchange *)
  if k is k1.+1 then codd_jump r :: knuth_jump_rec m k1 (uphalf r).-1 else [::].
Fixpoint knuth_exchange m : network (`2^ m) :=
  if m is m1.+1 then
    neodup (knuth_exchange m1) ++ ceswap :: knuth_jump_rec (`2^ m) m1 ((`2^
m1).-1)
  else [::].
```

with `sorting_batcher` and `sorting_knuth_exchange` proved. Note that “Batcher” names two different networks here; `djbsort`’s portable `sort.c` implements the **exchange** sorter, not the **merge** sorter.

Alongside the recursive `knuth_exchange`, the development also carries an **imperative** version, `iknuth_exchange`, a `seq`-level program with three nested loops `iter1/iter2/iter3` performing swaps. The two are proved to sort independently of one another; nothing connects them. Which of the two a proof picks up determines its whole shape. The `int32` verification originally took the imperative one and left the network world to do it; it now takes the recursive one and stays inside (§4.3).

3 Shared algebra NEW

These are generic: they mention no program. They live in `code/common/nalgebra.v` and are used by both tracks.

3.1 Index pairs as networks

A program emits a **list of comparators**. To reason about it as a network:

```

Definition oconn (n : nat) (ab : nat * nat) : option (connector n) :=
  obind (fun i => omap (fun j => cswap i j) (insub ab.2)) (insub ab.1).
Definition pnet (n : nat) (ps : seq (nat * nat)) : network n := pmap (oconn n) ps.

Lemma pnet_cons (n x y : nat) ps (xn : x < n) (yn : y < n) :
  pnet n ((x, y) :: ps) = cswap (Sub x xn) (Sub y yn) :: pnet n ps.

```

`pnet` produces one connector per **single** comparator, so it always yields a fully sequential network. Recovering parallel stages from it is what §3.4 is about.

3.2 Flip-free connectors

```

Definition cnoflip n (c : connector n) : bool := [forall i, ~~ cflip c i].
Definition nnoflip n (nt : network n) : bool := all (@cnoflip n) nt.

```

This is the second conjunct of the note’s `ctransp`, without its requirement that links be $i \pm 1$ — a requirement `codd_jump r` violates for $r > 1$. Closure lemmas: `cnoflip_odd_jump`, `cnoflip_eomerge`, `nnoflip_neodup`, `nnoflip_neotile`.

3.3 Commutation

```

Definition cdisjoint m (c1 c2 : connector m) : Prop :=
  forall i : 'I_m, (clink c1 i == i) || (clink c2 i == i).

Lemma cfun_comm m (c1 c2 : connector m) t :
  cdisjoint c1 c2 -> cfun c1 (cfun c2 t) = cfun c2 (cfun c1 t).
Lemma nfun_nswap m (n1 n2 : network m) (c1 c2 : connector m) t :
  cdisjoint c1 c2 ->
  nfun (n1 ++ c1 :: c2 :: n2) t = nfun (n1 ++ c2 :: c1 :: n2) t.

```

Two connectors that share no line commute, so adjacent disjoint connectors in a network may be exchanged. This is what licenses reordering a comparator sequence. Brought down to the list level, with the `pnet` bookkeeping done once and for all:

```

Lemma nfun_pnet_swap n (ps qs : seq (nat * nat)) a b c e u :
  a < n -> b < n -> c < n -> e < n ->
  a != c -> a != e -> b != c -> b != e ->
  nfun (pnet n (ps ++ (a, b) :: (c, e) :: qs)) u
  = nfun (pnet n (ps ++ (c, e) :: (a, b) :: qs)) u.

```

via `cdisjoint_cswap` and `ord_sub_neq`. Any claim that a program emits its comparators “in a different but equivalent order” reduces to repeated use of this, through three derived moves: `nfun_pnet_moveL` (move one comparator left past a block it is disjoint from), `nfun_pnet_moveL_block` (move a whole block), and `nfun_pnet_heads_first` (run every index’s first block before any of the second blocks — with the sortedness and membership hypotheses the disjointness genuinely needs, since for arbitrary $a < b$ it fails).

3.4 Reading a connector back as comparators

The inverse of `pnet`: which comparators does a connector perform?

```

Definition cpairs n (c : connector n) : seq (nat * nat) :=
  pmap (fun i : 'I_n =>
    if (i < clink c i)%N then Some (nat_of_ord i, nat_of_ord (clink c i))
    else None)
    (enum 'I_n).
Definition nstages n (nt : network n) : seq (nat * nat) :=
  flatten (map (@cpairs n) nt).

Lemma nfun_pnet_cpairs n (c : connector n) t :
  cnoflip c -> nfun (pnet n (cpairs c)) t = cfun c t.
Lemma nfun_pnet_nstages n (nt : network n) t :
  nnoflip nt -> nfun (pnet n (nstages nt)) t = nfun nt t.

```

`nfun_pnet_cpairs` is the bridge between the sequential and the parallel view: a connector's comparators are pairwise disjoint, because `clink` is an involution, so performing them one at a time is the same as performing the stage. It is what makes a flat program trace comparable with a structured network at all.

The proof is worth a remark, because the obvious route is unpleasant. Case splitting on a line and cutting `enum 'I_n` around the comparator that touches it degenerates into list surgery. Instead one generalises over the list l of **generating** lines — those j with $j < \text{clink } cj$, which is what `cpairs` scans the enumeration for — and describes the whole run in closed form:

```

tnth (nfun (pnet n [seq (j, clink c j) | j <- l]) u) i
= if i \in l then min (u i) (u (clink c i))
  else if clink c i \in l then max (u (clink c i)) (u i)
  else u i

```

Each induction step then only checks the three positions the head comparator can reach. The fact that makes this work is that a generator's partner is never itself a generator.

Also here: `cpairs_bounded`, and `tnth_nfun_pnet_avoid` (a line touched by none of the comparators keeps its value).

3.5 Iterated deinterleave

`neodup` distributes over concatenation:

```

Lemma neodup_cat n (n1 n2 : network n) :
  neodup (n1 ++ n2) = neodup n1 ++ neodup n2.

```

which lets a recursive network be flattened. Iterating `neodup` naively is painful, though: it goes `network m -> network (m + m)`, so j iterations land in a tower $(m + m) + (m + m) + \dots$ and every equation drowns in casts. Since `e2n` is defined by doubling rather than by `expn`, the type 2^{j+1} is **definitionally** $2^j + 2^j$, so indexing the iteration by the exponent keeps the type a power and no cast is ever needed:

```

Fixpoint neotile q (net : network (2^q)) j : network (2^(j + q)) :=
  if j is j1.+1 then neodup (neotile net j1) else net.

```

with `neotile0`, `neotileS`, `neotile_cat`, `nnoflip_neotile`. This is the interleaved sibling of the AVX2 track's blocked `ntile` (§5.4); the two solve the same problem for the two ways of splitting an array.

Finally, the index-level counterpart of `nfun_eodup`, saying that the enumeration of `'I_(m + m)` is the sub-problem's enumeration with each line a expanded to the adjacent pair $2a, 2a + 1$:

```
Lemma iota_eocat m : iota 0 (m + m) = flatten [seq [:: a.*2; a.*2.+1] | a <- iota 0 m].
Lemma enum_ord_eocat m :
  enum 'I_(m + m) = flatten [seq [:: elift a; olift a] | a <- enum 'I_m].
```

3.6 Comparators under deinterleaving NEW

Combining the previous two sections: deinterleaving doubles every comparator, running (a, b) on the even lines and on the odd lines, as $(2a, 2b)$ and $(2a + 1, 2b + 1)$. Because the enumeration visits $2a$ immediately before $2a + 1$, the two copies come out **adjacent**, so the comparator list of a deinterleaved network is the original list with each entry expanded in place.

```
Definition pdup (ps : seq (nat * nat)) : seq (nat * nat) :=
  flatten [seq [:: (ab.1.*2, ab.2.*2); (ab.1.*2.+1, ab.2.*2.+1)] | ab <- ps].
Fixpoint pdupn j ps := if j is j1.+1 then pdup (pdupn j1 ps) else ps.

Lemma cpairs_eodup n (c : connector n) : cpairs (ceodup c) = pdup (cpairs c).
Lemma nstages_neodup n (nt : network n) : nstages (neodup nt) = pdup (nstages nt).
Lemma nstages_neotile q (net : network (2^q)) j :
  nstages (neotile net j) = pdupn j (nstages net).
```

with `pdup_cat`, `pdup_flatten`, `pdup_pmap`, `neodupE` (`neodup nt is map ceodup nt`) and `clink_ceodup_e / _o`.

On the lists a program actually emits — ordered, in range — `pnet` and `nstages` are inverse (`nstages_pnet`, via `cpairs_cswap` and `pmap_single`), and the resulting network never flips (`nnoflip_pnet`). Hence

```
Lemma nfun_pnet_pdup n ps (t : (n + n).-tuple A) :
  all (okp n) ps -> nfun (pnet (n + n) (pdup ps)) t = nfun (neodup (pnet n ps)) t.
```

— the doubled comparator list runs the original list on the even lines and on the odd lines. This is what lets an induction on a comparator list pass through `nfun_eodup`.

Two groups of small generic lemmas support this and §4.5. First, pushing `map`, `filter` and `pmap` through `flatten` in an explicit form — `map_flatten_seq`, `filter_flatten_seq`, `pmap_flatten_seq`, `flatten_map_filter`. The library's `map_flatten` reassociates into the `[seq _ | x <- s, y <- x]` notation, which `map_comp` can then no longer see through, so the explicit versions are what let two flattened comprehensions be compared termwise. Second, two arithmetic facts,

```
Lemma divn_double a p : 0 < p -> a.*2 %/ p.*2 = a %/ p.
Lemma divn_doubleS a p : 0 < p -> a.*2.+1 %/ p.*2 = a %/ p.
```

which say that doubling numerator and denominator leaves a quotient alone, with or without the odd offset the odd lines carry. That is exactly what makes a test of the form “bit k of the index is clear” survive deinterleaving.

3.7 Transpose, rows and columns, sign flips, tiling NEW

The AVX2 track built a second answer to “run this network everywhere” — the **blocked** one, where `neotile` above is the interleaved one — and it has now moved here from `sort_transpose.v`, unchanged. It is described in §5.1–§5.4, where it is used; nothing in it mentions a program.

4 The `int32` track

`sort.c` implements Knuth 5.2.2M as a flat loop: `p` descends from `top` by halving, and for each base position the whole distance cascade is run at once.

4.1 The comparator model

The `C` is transcribed as the exact list of comparators it emits, in order.

```
Definition me_top (n : nat) : nat := top_loop n 1 n. (* sort.c's `top` *)
Definition level_pairs (N p d : nat) (b : bool) : seq (nat * nat) :=
  [seq (i, i + d) | i <- [seq i <- iota 0 N | (i + d < N) && (odd (i %/ p) ==
b)]]].
Definition casc_pairs (N top p : nat) : seq (nat * nat) :=
  flatten [seq [seq (j + p, j + r) | r <- [seq r <- halves top top | (p < r) && (j
+ r < N)]]
          | j <- [seq j <- iota 0 N | ~~ odd (j %/ p)]]].
Definition me_pairs (n : nat) : seq (nat * nat) :=
  flatten [seq level_pairs n p p false ++ casc_pairs n (me_top n) p | p <- halves
(me_top n) (me_top n)].

Definition int32_sort_network (n : nat) : network n := pnet n (me_pairs n).
```

The cascade is grouped **by position, then by distance** — not by distance, as a transcription of step M3 would naturally give. Reproducing that exact order is what makes `me_pairs n` equal to the trace of the `C`, and it is also the source of every difficulty below.

4.2 Arbitrary n

Three facts reduce the general case to a power of two: `me_pairs_bounded` (every emitted pair is in range), `me_pairs_prune` (`me_pairs n` is `me_pairs` at the next power of two, filtered to the lines below n), and

```
Lemma sorting_pnet_prune (N n : nat) (ps : seq (nat * nat)) :
  n <= N -> all (fun ab => ab.1 < ab.2 < N) ps ->
  pnet N ps \is sorting -> pnet n [seq ab <- ps | ab.2 < n] \is sorting.
```

the last being a general 0-1-principle statement: pruning a sorting pair-network to its low lines still sorts.

4.3 The power-of-two case NEW

`int32_knuth.v` proves it in the style of the AVX2 track: stay inside `network` and prove an equation between networks.

```
Theorem nfun_int32_knuth m t :
  nfun (int32_sort_network (`2^ m)) t = nfun (knuth_exchange m) t.
```

Sorting is then immediate from `sorting_knuth_exchange`. The induction on m has three ingredients.

The sweep splits. `nfun_me_pairs_split`: `sort.c`'s sweep at 2^{m+1} is its sweep at 2^m with every comparator doubled, followed by the $p = 1$ block. The $p \geq 2$ blocks double by `level_pairs_double` (a list identity) and by `nfun_casc_pairs_double`. The latter cannot be a list identity: the doubled sweep runs a whole chain on the even copy of a position and only then the whole chain on the odd copy, whereas `pdup` alternates per comparator. One copy lands entirely on even lines and the other entirely on odd ones, so every cross pair is disjoint and `nfun_pnet_mix` crosses the gap.

The network splits the same way. `nstages_knuth_exchanges`, and the merge stage's connectors turn out to be `sort.c`'s blocks rather than merely to correspond to them — `ceswap` is the base pass at distance 1 (`cpairs_eswap`), `codd_jump` is the distance- r pass on the odd lines (`cpairs_odd_jump`). `nfun_pnet_pdup` then turns the doubled comparator list into a deinterleaved network, so the induction hypothesis applies through `nfun_eodup`.

The cascade transposes. `nfun_casc_kjumps`: `sort.c` runs the cascade position-major, for each even position the whole distance chain; the network runs it distance-major, for each distance all positions. Peeling the largest distance off every chain and moving those comparators to the front turns one into the other, and each move is legitimate because a later position's large-distance comparator shares no line with an earlier position's smaller-distance ones — parity rules out three of the four coincidences, and $a + r < b + 2^{k+1}$ (from $a < b$ and $r \leq 2^k$) rules out the fourth. `nfun_pnet_heads_first` performs the move, `heads_level` identifies the comparators pulled out as one of `sort.c`'s base passes (the cascade indexes them by the even position j , the base pass by the odd line $j + 1$; the two are matched through `iota_eocat`).

4.4 Results

```
Theorem sorting_int32_sort_network n : int32_sort_network n \is sorting.
```

closed under the global context. What is **not** proved is that the C really emits `me_pairs n`; that is stated as an explicit assumption:

```
Parameter sortc_trace : nat -> seq (nat * nat).
Axiom sortc_faithful : forall n, sortc_trace n = me_pairs n.
```

4.5 Why the block order is not the obstacle NEW

The reason the network route was long thought impossible is that `knuth_exchange` deinterleaves and recurses innermost-first, while `me_pairs` — like `sort.c` — is a flat loop $p = \text{top}, \frac{\text{top}}{2}, \dots, 1$ outermost-first. But pushing `neodup_cat` through the unfolded recursion gives

$$\text{knuth_exchange}(m) = \text{neodup}^{m-1} \text{merge}_1 ++ \dots ++ \text{merge}_m$$

where merge_k is `ceswap` followed by the `knuth_jump_rec` chain on 2^k lines. Each `neodup` doubles distances, so the blocks come out in **decreasing** distance — exactly the order the flat sweep visits p in. The two sides line up block by block; what is left inside a block is the cascade transpose, which §4.3 settles.

`kjumps n k` is the flat counterpart of the jump chain, whose distances are $2^k - 1, 2^{k-1} - 1, \dots, 1$: each step halves via `(uphalf r).-1`, and on numbers of that shape it lands exactly on the next one down (`uphalf_e2n_pred`), all of them odd and positive (`odd_e2n_pred`).

5 The `avx2` track

Here the program is not a list of comparators at all: it is a loop nest over a vector width, in which a comparator's **distance** decides whether it becomes a vector min/max or a lane shuffle. The network is the same for every width, so the proof goes straight to the network.

The constructions of §5.1–§5.4 were built for this track but are generic, and now live in `nalgebra.v` (§3.7); `sort_transpose.v` keeps only what is actually about `sort_transpose.ml`.

5.1 Transposition

```

Definition trp (i : 'I_(m * m)) : 'I_(m * m) := ...      (* index transpose *)
Definition ttr (t : (m * m).-tuple A) : (m * m).-tuple A := ...
Definition rsh (t : (m * m).-tuple A) : m.-tuple (m.-tuple A) := ...  (* reshape *)
Definition fla (M : m.-tuple (m.-tuple A)) : (m * m).-tuple A := ...
Definition cconj (c : connector (m * m)) : connector (m * m) := ...
Definition nttr (net : network (m * m)) : network (m * m) := map cconj net.

Lemma nfun_nttr net t : nfun (nttr net) t = ttr (nfun net (ttr t)).

```

Conjugating a network by the transpose: this is what turns “sort the columns” into “transpose, sort the rows, transpose back”, which is what the AVX2 code actually does.

5.2 Rows and columns

```

Definition crow (c : connector m) : connector (m * m) := ...
Definition nrows (net : network m) : network (m * m) := map crow net.
Definition ncols (net : network m) : network (m * m) := nttr (nrows net).

```

with `nfun_nrows`, `nfun_ncols`, `nrows_sorted`, `ncols_sorted`.

5.3 Sign flips

The code obtains descending order by exclusive-oring with a mask of 0/−1 words rather than by using a different comparator.

```

Definition tflip (msk : (m * m).-tuple bool) (t : (m * m).-tuple A) := ...
Definition ctflip (msk : ...) (c : connector (m * m)) : connector (m * m) := ...
Definition ntflip (msk : ...) (N : network (m * m)) : network (m * m) := ...
Definition mask_luni (msk : (m * m).-tuple bool) : Prop := ...  (* lane-uniform *)

Lemma nfun_conj msk cc_net cw_net : ...
Lemma nfun_ntflip_conj msk cc_net : ...

```

Conjugation by a sign flip is the second of the two conjugations the track needs; together with `nttr` they are what let one uniform network stand for the code’s asymmetric-looking phases.

5.4 Tiling and reshape

This is the group that has no counterpart in the foundation, and the one the `int32` track turned out to want too.

```

Fixpoint ntile (net : network (`2^ q)) j : network (`2^ (j + q)) := ...
Definition arsh j (t : (`2^ (j + q)).-tuple A) : (`2^ j).-tuple ((`2^ q).-tuple A) := ...
Definition afla j (M : (`2^ j).-tuple ((`2^ q).-tuple A)) : (`2^ (j + q)).-tuple A := ...
Definition sqpow : network (`2^ (q + q)) := ...
Definition sqcast (u : (`2^ (q + q)).-tuple A) : (`2^ q * (`2^ q)).-tuple A := ...

```

```

Lemma nfun_ntile_arsh net j t : ...      (* blockwise semantics *)
Lemma ntile_ntile p net q j : ...      (* nested tiling collapses *)
Lemma nfun_tile_sqpow_flat j t : ...

```

`ntile` runs a block network on every block of a larger array, and, exactly as for `neotile` (§3.5), is indexed by the exponent so that no cast appears. `ntile_ntile` collapses nested tilings — the operation that converts a doubly nested loop into a single flat one.

5.5 The sort itself, and its target

```

Definition tmerge_avx2 (b : bool) m (t : (`2^ m).-tuple A) := ... (* the code *)
Fixpoint tsort (b : bool) k : (`2^ k).-tuple A -> (`2^ k).-tuple A := ...
Fixpoint pbsort (b : bool) k : network (`2^ k) := ... (* the target *)

Lemma sorting_pbsort k : pbsort false k \is sorting.
Lemma tsortE b k t : tsort b k t = nfun (pbsort b k) t.
Lemma tmerge_avx2P b m t : ...

```

The target is the **periodic** bitonic network `pbsort`, not the plain `bfsort`; identifying it correctly was itself a step in the development. Padding to a power of two is handled generically by `nfun_pad`, `nfun_pad_sorted`, `nfun_pad_perm` in `sort_generic.v`.

5.6 Results

```

Theorem tsort_avx2_pbsort k t : tsort tmerge_avx2 false t = nfun (pbsort false k) t.
Corollary avx2_sort_sorted k t : ...

```

with `avx2_sort_perm`, `avx2_sort_pad_sorted`, `avx2_sort_pad_perm`, all closed under the global context. As for `int32`, that the OCaml/C source really runs this network is not formalised; here it is checked empirically instead of assumed, by `ml/trace_check.ml`, which compares the comparator trace of the source against a transcription of the Rocq definitions for every power-of-two width.

6 Status and layout

6.1 What is proved

Result	Status	Meaning
<code>sorting_int32_sort_network</code>	closed	<code>sort.c</code> 's exact network sorts, every n
<code>sortc_faithful</code>	axiom	the C emits that exact trace
<code>tsort_avx2_pbsort</code>	closed	the AVX2 sort computes <code>pbsort</code>
<code>avx2_sort_sorted, _perm</code>	closed	hence it sorts and permutes
<code>nfun_pnet_cpairs</code>	closed	sequential comparators = parallel stage
<code>nstages_neodup</code>	closed	deinterleaving doubles every comparator
<code>level_pairs_double</code>	closed	the base pass survives deinterleaving
<code>cpairs_odd_jump</code>	closed	a merge connector is one of <code>sort.c</code> 's blocks
<code>nstages_knuth_exchangeS</code>	closed	one unfolding step, as comparator lists
<code>nfun_pnet_swap</code>	closed	disjoint comparators may be exchanged
<code>nfun_casc_kjumps</code>	closed	the cascade transpose
<code>nfun_me_pairs_knuth</code>	closed	flat sweep = deinterleaved recursion
<code>nfun_int32_knuth</code>	closed	<code>sort.c</code> 's network computes <code>knuth_exchange</code>

Everything listed is closed under the global context. `sortc_faithful` is the only assumption anywhere in the development, and it is about the C source, not about the algorithm.

6.2 Files

<code>code/common/more_tuple.v</code>	tuple and index helpers, the <code>e2n</code> power notation
<code>code/common/nsort.v</code>	connectors, networks, <code>sorting</code> , binary combinators
<code>code/common/nbitonic.v</code>	half cleaners, <code>bsort</code> , <code>bfsort</code>
<code>code/common/nalgebra.v</code> NEW	the shared algebra of §3, including §5.1–§5.4
<code>code/portable4/proof/nbjsort.v</code>	<code>knuth_exchange</code> and <code>iknuth_exchange</code>
<code>code/portable4/proof/int32_network.v</code>	<code>me_pairs</code> , the reduction facts
<code>code/portable4/proof/int32_knuth.v</code> NEW	the power-of-two case, §4.3
<code>code/portable4/proof/int32_sort.v</code>	the final theorem, <code>sortc_faithful</code>
<code>code/avx2/proof/sort_generic.v</code>	<code>gnet</code> , <code>pbsort</code> , padding
<code>code/avx2/proof/sort_transpose.v</code>	§5.5–§5.6: the square block, the phases, the capstone

6.3 A remark on what was missing

Both tracks needed the same thing and neither could get it from the foundation: a way to say “run this network everywhere”. The AVX2 track built it for blocks (`ntile`) and got a clean development; the `int32` track did not build it and worked around it on lists instead, in a 633-line file that reused nothing and that nothing could reuse. That file is gone: with the shared algebra of §3 in place — and `neotile`, the same idea for the other way of splitting an array — the `int32` proof is half the size and every piece of it is generic. The AVX2 track’s own toolkit has moved to the same place, so `sort_transpose.v` is down from 1345 lines to 717 and what remains in it is only what is about the program.